IT342 – Required System Features (Final Implementation)
Your approved system concept remains the same.
The following features are now **MANDATORY** for your implementation.
All features below must be working during Final Defense.

1. **Authentication & Security (Required)**
   Your system must implement:
   - User Registration
   - User Login
   - JWT Authentication
   - Logout
   - Password hashing (BCrypt)
   - Protected routes/pages
   - /me endpoint (get current authenticated user)
2. **Role-Based Access Control (Required)**
   - Minimum **2 user roles**
   - API-level role restriction
   - UI-level role restriction

   Example:
   - Admin can manage data
   - Regular user has limited access
3. **Core Business Module (Required)**
   Your system must include:
   - At least **1 major entity (Except User entity**
   - Full CRUD operations
   - Proper validation

   Example:
   - Orders → OrderItems
   - Projects → Tasks
   - Users → Appointments
4. **System Integration Features (ALL REQUIRED)**
   Your system must implement **ALL the following integrations**:

   4.1. External API Integration
   - Must consume a real public API
   - Must be used in a meaningful feature
   - Data must be displayed in your system

   4.2. Google OAuth Login (or any other Authentication server/Social Login)
   - Login using Google or other
   - User saved/linked in database
   - Must generate your own JWT after OAuth login

   4.3. File Upload (depends on the business domain)
   - Upload file (image/pdf/etc.)
   - File stored on server
   - File linked to database record
   - User can view/download it

   4.4. Payment Gateway Integration (Sandbox Only)
   - Payment integration is required **only if it makes sense for your system**.

- Use real payment provider (Test Mode)
- Record payment results in database
- Handle success and failure

No simulated payment allowed.

4.5. Real-Time Feature (Optional – Bonus)

Use either:

- WebSocket (recommended), OR
- Polling (auto-refresh every few seconds)

Must demonstrate live updates between users.

4.6. Email Sending (SMTP)

Must send:

- One account-related email (e.g., welcome, verification)
- One system notification email (e.g., receipt, approval)

Console print is NOT accepted.

4.7. Backend

- Spring Boot (Java 17+)
- Spring Security + JWT
- JPA/Hibernate
- MySQL or PostgreSQL or MongoDB or Firebase

Backend must be custom-built using Spring Boot.
No Backend-as-a-Service.

4.8. Web Application

- ReactJS
- Must consume the Spring Boot REST API
- Must implement protected routes

No mock or hardcoded data.

5. Database Requirements

- Minimum 5 database tables
- Proper relationships (One-to-Many or Many-to-One)
- No plain-text passwords
- Proper normalization (avoid duplicate data)
- Use DTOs (do not expose password in API responses)

6. Architecture Requirements

Your backend must follow a clear System Architecture Pattern.

Examples:

- Layered Architecture
- Clean Architecture
- Modular structure

Minimum requirement:

- Controller layer
- Service layer
- Repository layer
- DTOs
- Security configuration
- Global exception handling

You must include:

- Architecture diagram in documentation

- Short explanation of the pattern you used

Must use:

- Proper HTTP status codes
- RESTful endpoint naming

7. **Mobile Application Requirement (REQUIRED)**

Your system must include a working Android mobile application that integrates with your backend API.

7.1. Technology Requirement (STRICT)

The mobile application must:

- Use **Android Kotlin**
- Use **XML-Based UI Layouts**
- Use **Android API Level 34 (Android 14)**
- Use **Retrofit for API integration**
- Connect to the SAME backend used by the web application

Jetpack Compose is **NOT** allowed.

Mock data is **NOT** allowed.

7.2. Authentication Integration

- Login using backend JWT authentication
- Store JWT securely
- Send JWT in Authorization header
- Access protected endpoints

7.3. Core Module Consumption

- Display data from the Core Business Module
- Access at least 5 protected API endpoint
- Handle error responses properly

7.4. Role Awareness

- UI must adjust based on user role
- Unauthorized actions must not be allowed

7.5. Proper API Integration

- Use Retrofit (or equivalent)
- No hardcoded/mock data
- Must connect to actual backend server

8. **Application Name**

Effective application names are short, memorable, and often reflect the core functionality Ideal names consist of one or two words, are easy to pronounce, and often use suffixes like -ify, -io, or -ly to create unique, brandable, and modern-sounding titles.

9. **Required Repository Structure**

IT342_<AppName>_<Section>_<Lastname>

```
├── backend/
├── web/
├── mobile/
├── docs/
└── README.md
```